

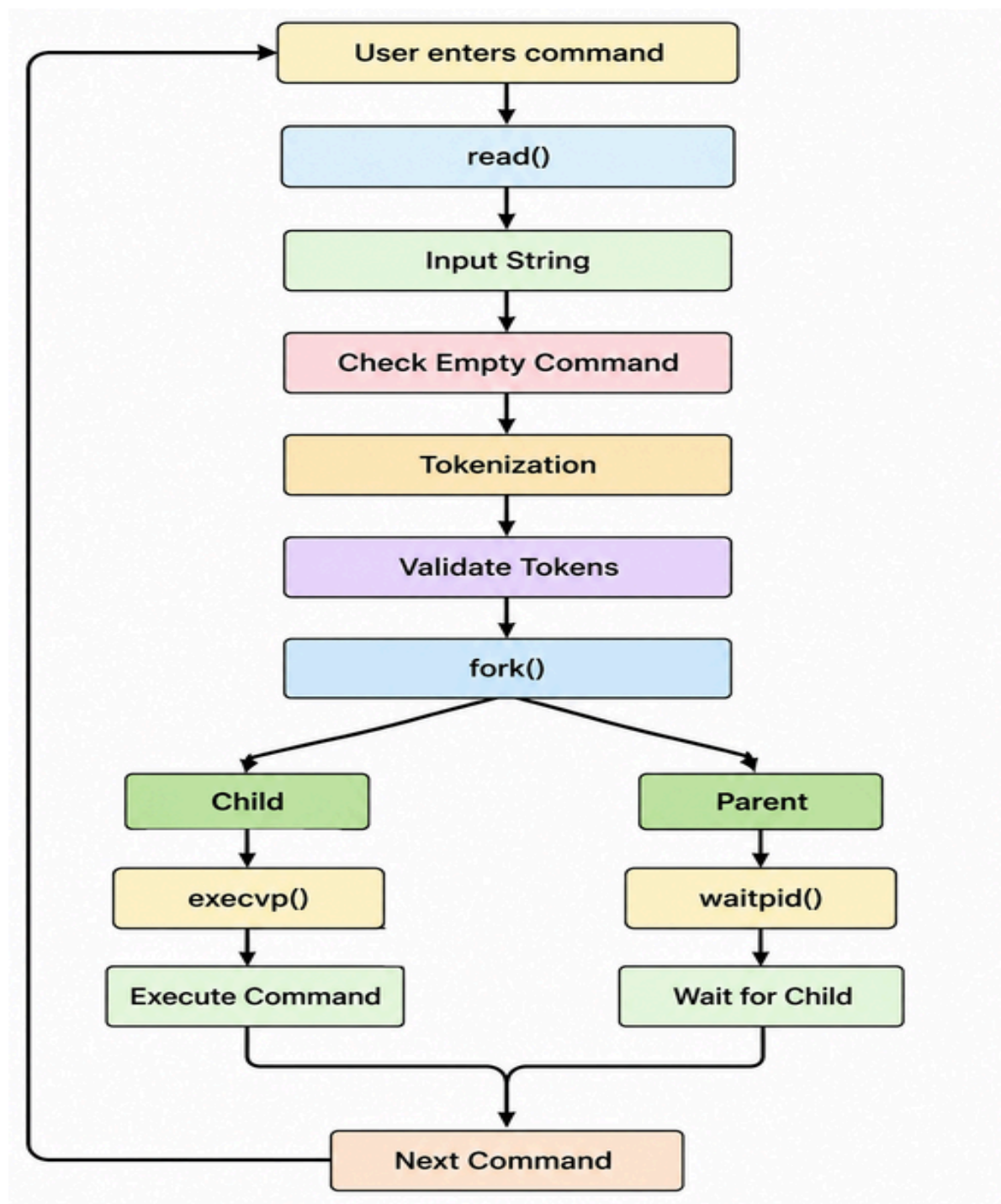
Refer the following link to understand the Parsing and Lexical analysis

<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Flow Chart:




```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024
#define MAX_TOKENS 64

int main() {
    char buffer[BUFFER_SIZE];
    char *tokens[MAX_TOKENS];

    while (1) {
        // 1. Display Prompt
        write(1, "myShell> ", 9);

        // 2. Read user input using read() system call
        memset(buffer, 0, sizeof(buffer));
        int bytes_read = read(0, buffer, sizeof(buffer) - 1);

        if (bytes_read <= 0) {
            printf("\nExiting shell...\n");
            break;
        }

        // Remove trailing newline character
        if (buffer[bytes_read - 1] == '\n') {
            buffer[bytes_read - 1] = '\0';
        }

        // 3. Check for Empty Command
        if (strlen(buffer) == 0) {
            continue;
        }

        // Check for exit
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting shell.\n");
            break;
        }

        // 4. Tokenization (Splitting input into tokens)
        int token_count = 0;
        char *token = strtok(buffer, " \t\r\n");

        while (token != NULL && token_count < MAX_TOKENS - 1) {
            tokens[token_count++] = token;
            token = strtok(NULL, " \t\r\n");
        }
        tokens[token_count] = NULL; // NULL-terminate the array

        // 5. Validate Tokens & Debug Parsing Output
        if (token_count == 0) {
            continue;
        }

        printf("--- Parsing Result ---\n");
        for (int i = 0; i < token_count; i++) {
            printf("Argument %d: %s\n", i + 1, tokens[i]);
        }

        // 6. Fork and Execute
        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
        } else if (pid == 0) {
            // Child process
            printf("Child PID: %d\n", getpid());
            if (execvp(tokens[0], tokens) < 0) {
                perror("Command execution failed");
                exit(EXIT_FAILURE);
            }
        } else {
            // Parent process
            int status;
            waitpid(pid, &status, 0);
            printf("Command execution completed.\n\n");
        }
    }

    return 0;
}

```

```
Ubuntu × + ▾
speed@DESKTOP-P4G181S: ~$ nano task1_parser.c
speed@DESKTOP-P4G181S: ~$ gcc task1_parser.c -o task1_parser
./task1_parser
myShell> Hello
--- Parsing Result ---
Argument 1: Hello
Child PID: 938
Command execution failed: No such file or directory
Command execution completed.

myShell> Time
--- Parsing Result ---
Argument 1: Time
Child PID: 939
Command execution failed: No such file or directory
Command execution completed.

myShell> End
--- Parsing Result ---
Argument 1: End
Child PID: 940
Command execution failed: No such file or directory
Command execution completed.

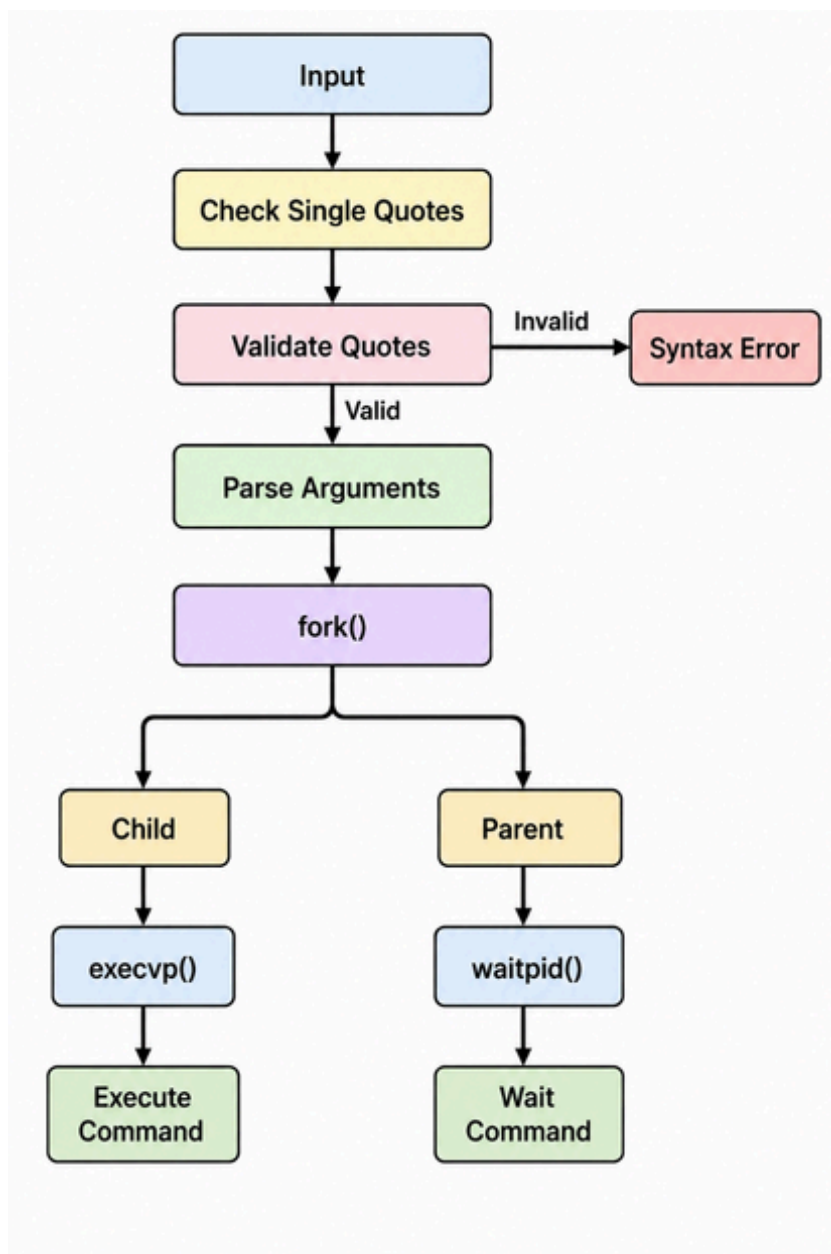
myShell> echo Hello
--- Parsing Result ---
Argument 1: echo
Argument 2: Hello
Child PID: 952
Hello
Command execution completed.

myShell> echo Operating Systems Lab
--- Parsing Result ---
Argument 1: echo
Argument 2: Operating
Argument 3: Systems
Argument 4: Lab
Child PID: 953
Operating Systems Lab
Command execution completed.
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210

Hello

Command execution completed.

Case2:

```
myShell> echo 'Hello World
```

Syntax Error: Unmatched single quote.

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

// Custom parser to handle single quotes
int parse_single_quotes(char *input, char **args, int *arg_count) {
    int i = 0, j = 0;
    int in_single_quote = 0;
    char current_token[MAX_INPUT];
    int token_len = 0;

    *arg_count = 0;

    while (input[i] != '\0') {
        char c = input[i];

        if (c == '\'') {
            in_single_quote = !in_single_quote; // Toggle quote state
        } else if (c == ' ' && !in_single_quote) {
            // Delimiter reached outside quotes
            if (token_len > 0) {
                current_token[token_len] = '\0';
                args[*arg_count] = strdup(current_token);
                (*arg_count)++;
                token_len = 0;
            }
        } else {
            // Add regular characters to the token
            current_token[token_len++] = c;
        }
        i++;
    }

    // Check for unmatched quote error
    if (in_single_quote) {
        printf("Syntax Error: Unmatched single quote.\n");
        return -1;
    }

    // Save final token if present
    if (token_len > 0) {
        current_token[token_len] = '\0';
        args[*arg_count] = strdup(current_token);
        (*arg_count)++;
    }

    args[*arg_count] = NULL;
    return 0;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int arg_count = 0;

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgetc(input, sizeof(input), stdin) == NULL) {
            printf("\nExiting...\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) continue;
        if (strcmp(input, "exit") == 0) break;

        // Parse with quote handling
        if (parse_single_quotes(input, args, &arg_count) != 0) {
            continue; // Syntax error occurred
        }

        if (arg_count == 0) continue;

        // Display parsed result
        printf("Parsing Result:\n");
        for (int i = 0; i < arg_count; i++) {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        // Fork & Execute
        pid_t pid = fork();
        if (pid < 0) {
            perror("Fork failed");
        } else if (pid == 0) {
            printf("Child PID: %d\n", getpid());
            if (execvp(args[0], args) < 0) {
                perror("Execution error");
                exit(EXIT_FAILURE);
            }
        } else {
            waitpid(pid, NULL, 0);
            printf("Command execution completed.\n\n");
        }

        // Free allocated memory
        for (int i = 0; i < arg_count; i++) {
            free(args[i]);
        }
    }
}

```

```
return 0;  
}
```

```
Ubuntu × + ▾  
speed@DESKTOP-P4G181S:~$ nano task2_single_quotes.c  
speed@DESKTOP-P4G181S:~$ gcc task2_single_quotes.c -o task2_single_quotes  
./task2_single_quotes  
myShell> echo Operating Systems Lab  
Parsing Result:  
Argument 1: echo  
Argument 2: Operating  
Argument 3: Systems  
Argument 4: Lab  
Child PID: 1077  
Operating Systems Lab  
Command execution completed.  
  
myShell> exit  
speed@DESKTOP-P4G181S:~$ █
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:

Case1:

```
myShell> echo "Hello $USER"
```

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

Hello raghu

Child process completed.

Case2:

myShell> echo "Hello World"

Syntax Error: Unmatched double quote.

```
GNU nano 2.9.3 /
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 256
#define MAX_LEN 64

// Function to expand environment variables (HOME)
void expand_variables(char *input, char *output) {
    int i = 0, j = 0;
    while (input[i] != '\0') {
        if (input[i] == '$' && (input[i+1] == 'H' && input[i+1] == 'O' && input[i+1] == 'M' && input[i+1] == 'E')) {
            // Expand HOME variable
            char *home = getenv("HOME");
            if (home != NULL) {
                output[j++] = home[0];
                j++;
            }
        } else {
            output[j++] = input[i++];
        }
    }
    output[j] = '\0';
}

// Parse double quotes
int parse_double_quotes(char *input, char **args, int *arg_count) {
    int i = 0;
    while (input[i] != '\0') {
        if (input[i] == '\"') {
            // Start of double quote
            i++;
            // Find the end of the double quote
            while (input[i] != '\"') {
                i++;
            }
            // Extract the token
            char *token = malloc(i - 1 + 1);
            if (token != NULL) {
                strncpy(token, input + 1, i - 1);
                token[i - 1] = '\0';
                args[*arg_count] = token;
                (*arg_count)++;
                free(token);
            }
            // Skip the double quote
            i++;
        } else {
            // Not a double quote
            i++;
        }
    }
    return 0;
}

// Main function
int main() {
    char *args[MAX_ARGS];
    int arg_count = 0;

    // Parse the input
    if (parse_double_quotes(input, args, &arg_count) != 0) {
        printf("Syntax Error: Unmatched double quote.\n");
        return -1;
    }

    // Execute the command
    if (execvp(args[0], args) < 0) {
        perror("Execution error");
        exit(EXIT_FAILURE);
    }

    // Wait for the child process to complete
    waitpid(-1, NULL, 0);

    // Print the output
    printf("Child process completed.\n\n");

    // Free memory
    for (int i = 0; i < arg_count; i++) {
        free(args[i]);
    }

    return 0;
}
```

```
pid_t pld = fork();
if (pld < 0) {
    perror("Fork failed");
} else if (pld == 0) {
    printf("Child Process\nPID = %d\n", getpid());
    if (execvp(args[0], args) < 0) {
        perror("Execution error");
        exit(EXIT_FAILURE);
    }
} else {
    waitpid(pld, NULL, 0);
    printf("Child process completed.\n\n");
}

// Free Memory
for (int i = 0; i < arg_count; i++) {
    free(args[i]);
}

return 0;
}
```

```
speed@DESKTOP-P4G181S:~$ nano task3_double_quotes.c
speed@DESKTOP-P4G181S:~$ gcc task3_double_quotes.c -o task3_double_quotes
./task3_double_quotes
myShell> echo Operating Systems Lab
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Operating
Argument 3: Systems
Argument 4: Lab
Child Process
PID = 1121
Operating Systems Lab
Child process completed.
```

```
myShell> echo 'Hello World'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: 'Hello
Argument 3: World'
Child Process
PID = 1183
'Hello World'
Child process completed.

myShell> █
```